stellaris 舰船索敌权重计算公式

古明地瓜 (KKoishi_)：反编译分析整理

地砖 & 清流蛋糕 rt：辅助数据验证

0 约定

本文基于 Stellaris **4.5.2**（**Cygnus**，二进制 hash `32709cf254181b032b1b309a6ea09776`）的反编译产物 `stellaris_4.5.2_source.cpp` 整理，文中行号与结构偏移均以该版本为准。

索敌权重**不是每 tick 重算**：舰船已有有效主目标（同星系、存活、非 disabled/cloak）时，只按 (TotalTicks + shipID) mod (10 或 50) 错峰重算一次，只有" 没有有效主目标" 时才立刻重算，详见 §3。

约定如下出现的命中、闪避、伤害等属于**舰船属性**的量均为经由修正计算后的量。

符号	含义
F	定点单位，$F = 100000$（游戏中 1.0 记作 100000）
$\lfloor x \rfloor$	C 风格定点整数除法（向零截断）；本文中出现的除法均为该运算
σ	候选目标的尺寸倍率 = shipSize[$+0x630$]，即脚本 `size_multiplier`
$\sigma_{\max}$	`COMBAT_SHIP_TARGETING_SIZE_MAX`
κ_{size}	`COMBAT_SHIP_TARGETING_SIZE_MULT`
a_{trk}	攻击方追踪字段（$attacker + 0x810$）
κ_{trk}	`COMBAT_SHIP_TARGETING_TRACKING_MULT`
e_{ev}	候选目标闪避字段（$candidate + 0x590$，存储为 $1.0 = 10^7$）
κ_{ev}	`COMBAT_SHIP_EVASION_TARGETING_WEIGHT`
T_{avoid}	`COMBAT_SHIP_EVASION_AVOID_THRESHOLD`
$\sigma_{\min}^{\text{avoid}}$	`COMBAT_SHIP_EVASION_AVOID_MIN_SHIP_SIZE`
d	候选距离字段（`SSinglePotentialShipData` $+0x30$）
κ_d，$d_{\min}$	`COMBAT_SHIP_DISTANCE_TARGETING_WEIGHT` / `…_MIN`
w_{tgt}	目标预置权重（`SSinglePotentialTargetData` $+0x30$），构成见 §1.5
h，θ_h	目标血量百分比 / `COMBAT_SHIP_LOW_HEALTH_THRESHOLD`
κ_h	`COMBAT_SHIP_LOW_HEALTH_TARGETING_WEIGHT`
κ_{un}，κ_{sb}	`COMBAT_SHIP_UNARMED_TARGETING_WEIGHT` / `…STARBASE…`
κ_r	`RANDOM_TARGETING_WEIGHT`
M_{cur}，M_{rec}	`SHIP_CURRENT_TARGET_MULTIPLIER` / `SHIP_RECENT_TARGET_PENALIZER`
w_{mult}	`MOD_SHIP_TARGET_WEIGHT_MULT`（`ModifierType` $0x6e$）
ρ	开战瞬间为每艘船掷一次的随机量，$\rho \in [0, F)$
f	目标焦点 `CShipTargeting::GetTargetFocus`（设计 growth stage $+0x128$）

$V_{\text{tgt}},\ V_{\text{tgt}}^{\max}$	目标**当前**船体（$+0xa68$）/ 船体上限（$+0xf20$）
$S_{\text{tgt}}, A_{\text{tgt}}$	目标当前护盾（$+0xa70$）/ 当前装甲（$+0xa78$）
m_h, m_s, m_a	武器模板的 `hull_damage` / `shield_damage` / `armor_damage` 倍率

1 Stage 1：通用舰船索敌权重 $W^{(\text{ship})}$

对应 `CShipTargeting::ComputePotentialTargetScore`（行 1577446）。前置条件：候选与自己同星系、可被攻击、且不处于跳跃/隐身位掩码中（不处于 FTL/隐身状态）；否则直接返回无效。

1.1 舰船尺寸项（作为基准）

$$\sigma' = \min(\sigma,\ \sigma_{\max}), \qquad u = \left\lfloor \frac{\sigma'\,\kappa_{\text{size}}}{F} \right\rfloor, \qquad W_1 = \left\lfloor \frac{F^2}{u+F} \right\rfloor \tag{1}$$

注意 W_1 是**赋值**而非累乘：当 $u \to 0$ 时 $W_1 \to F$（即 1.0），随尺寸增大单调递减，取值域为 $(0, F]$。

1.2 追踪 – 闪避项

$$\tau = \left\lfloor \frac{a_{\text{trk}}\,\kappa_{\text{trk}}}{F} \right\rfloor, \qquad \varepsilon = \left\lfloor \frac{e_{\text{ev}}\,F}{10^7} \right\rfloor, \qquad \Delta = \varepsilon - \tau \tag{2}$$

否决条件（整目标排除） 当 $\Delta > 0$ 且攻击方尺寸足够大、且候选目标**所在舰队**中存在一艘对攻击方” 可命中” 的舰船时，本函数直接返回 0，该候选直接被剔除:

$$\Delta > 0 \ \wedge\ \sigma_{\text{att}} \geq \sigma_{\min}^{\text{avoid}} \ \wedge\ \Delta \geq T_{\text{avoid}} \ \wedge\ \exists q \in \mathcal{F}_{\text{tgt}} :\ \varepsilon_q - \tau < T_{\text{avoid}} \implies \text{return } 0 \tag{3}$$

其中 $\mathcal{F}_{\text{tgt}}$ 是候选目标所在舰队的舰船集合，σ_{att} 是攻击方尺寸，ε_q 是舰船 q 的闪避归一值。

闪避惩罚 若未被上述条件否决且 $\Delta > 0$:

$$W_2 = \begin{cases} \left\lfloor \dfrac{W_1\,F}{\lfloor \Delta\,\kappa_{\text{ev}}/F \rfloor + F} \right\rfloor, & \Delta > 0 \\ W_1, & \Delta \leq 0 \end{cases} \tag{4}$$

1.3 距离项

$$D = \max\left(F - \left\lfloor \frac{d\,\kappa_d}{F} \right\rfloor,\ d_{\min} \right), \qquad W_3 = \left\lfloor \frac{W_2\,D}{F} \right\rfloor \tag{5}$$

1.4 目标预置权重、残血、无武装/星基

$$W_4 = \left\lfloor \frac{W_3\, w_{\text{tgt}}}{F} \right\rfloor \tag{6}$$

$$W_5 = \begin{cases} \left\lfloor \dfrac{W_4\, \kappa_h}{F} \right\rfloor, & h < \theta_h \\ W_4, & h \geq \theta_h \end{cases} \tag{7}$$

$$W_6 = \begin{cases} \left\lfloor \dfrac{W_5\, \kappa_{\text{un}}}{F} \right\rfloor, & \neg\,\text{CanDoDamage(tgt)} \\ \left\lfloor \dfrac{W_5\, \kappa_{\text{sb}}}{F} \right\rfloor, & \sigma_{\text{class}} = 10 \ \wedge\ \text{tgt} \neq \text{mainTarget} \\ W_5, & \text{otherwise} \end{cases} \tag{8}$$

其中 σ_{class} 是舰船类型枚举（10 表示恒星基地/巨构）。第二条分支的判定可以写作取反的合取语句：$\neg(\text{tgt} = \text{mainTarget} \wedge \text{mainTargetValid})$。

1.5 目标预置权重 w_{tgt} 的来源

w_{tgt} 不是脚本常量,而是由目标的舰船级修正与**开战瞬间的随机量**拼出来的(`CShip::GetTargetWeightFact`
行 3824970)：

$$w_{\text{tgt}} = \texttt{GetModifierValue}(\text{shipSize},\ 0x6e) + (F + \rho) \qquad \rho \in [0, F) \tag{9}$$

其中 $0x6e$ 是 `ModifierType` `MOD_SHIP_TARGET_WEIGHT_MULT`（注册于反编译 958356 行）；ρ 由 `CShip::OnCombatStarted`（行 3830406）用 `CRandom::GetFixedPointForMacro` 写入 $ship + 0xe88$，**整场战斗保持不变**，`OnCombatEnded` 时复位为 1.0。

因此即使两条船设计完全相同, 开战时会各掷一次 ρ, 把本场的" 偏爱" 固定下来 (实测两场分别是 1.326/1.486 与 1.561/1.379)。原版 `ship_target_weight_mult` 的来源包括船级定义与光环类组件(例如 `common/component_templates/00_dsc_auras.txt` 里的 0.50、`common/ship_sizes/29_nomads_dlc_`
里的 -0.5)。

1.6 随机扰动项

$$W^{(\text{ship})} = \begin{cases} \left\lfloor \dfrac{W_6\,(\lfloor r\,\kappa_r/F \rfloor + F)}{F} \right\rfloor, & \text{useRandom} \\ W_6, & \text{otherwise} \end{cases} \qquad r \in [0, F) \tag{10}$$

r 由 `CRandom::GetFixedPointForMacro` 给出,伪随机发生器生成一个随机数。`CShipTargeting::CalcBest`
默认传 `useRandom=true`。

2 Stage 2：武器索敌分数 $S^{(\text{weapon})}$

对应 `CWeaponComponentTargeting::ComputePotentialTargetScore`（行 1578737)。

2.1 期望伤害与命中率

武器层按下面的顺序算(全部在 `CWeaponComponentTargeting::ComputePotentialTargetScore` 里，行 1578737；调用顺序已用 objdump 核对)：

1. 把模板的 `SAttackInfo`（模板 $+0x11c8 \cdots + 0x124b$，约 $0x84$ 字节）拷到栈上。**区间编码为** $(\min,\ \max - \min)$：$+0x11c8$ 是最小伤害，$+0x11d0$ 是**跨度**，均值 $= \min +$跨度$/2$；
2. `CShip::ApplyModifier`（$0x2ab4a60$）就地改写这份拷贝——修正集合由**攻击方 + 控制方 + 目标（用目标 ship size 构造）**三方组成，所以用的是**修正加成后的伤害**，不是基础值；
3. `NCombatRules::CalcDamage`($0x22a2cb0$)拿这份" 已修正" 的 `SAttackInfo` 与目标的 `SDefenseInfo` 算出三个伤害分量；
4. `NCombatRules::CalcChanceToHit`（$0x22a3560$）算命中率。

$$\Sigma D = D_1 + D_2 + D_3, \qquad p = \mathrm{CalcChanceToHit}(\,\mathrm{evasion}_{\mathrm{tgt}},\ \tau_{\mathrm{w}},\ \mathrm{acc}_{\mathrm{w}}) \tag{11}$$

其中 τ_{w}、$\mathrm{acc}_{\mathrm{w}}$ 为武器模板的追踪率与命中率字段（$+0x11b0$ / $+0x11a8$）。
具体计算流程为两端都夹到 $[0, 1]$：

$$p = \min(F,\ \max(0,\ \mathrm{acc}_{\mathrm{w}} - \min(F,\ \max(0,\ \mathrm{evasion}_{\mathrm{tgt}} - \tau_{\mathrm{w}})))) \tag{12}$$

分层伤害与经验结论 `CalcDamage` 把伤害按**护盾 / 装甲 / 船体**三层分摊，每层再乘武器模板在 `weapon_components.csv` 里的 m_s / m_a / m_h 倍率，三项求和就是 ΣD。目标防御信息来自 `CShip::BuildDefenseInfo`(行 3813635),即 (当前护盾, 当前装甲, size_multiplier, 护盾硬化, 装甲硬化)。

实测结论：ΣD 不是随目标血量/EHP 单调，而是**随目标" 带护盾/装甲" 变大**——带 225 盾 +225 甲、240 船体的目标，ΣD 是同舰队 720 船体、0 盾 0 甲目标的 $1.58\times \ldots 2.65\times$（同一门武器、实测)。所以" 武器更想打带盾甲的船" 是真的，唯一有效的反制是**高闪避**（它压低 p)。

2.2 基础分数

$$S_0 = \left\lfloor \frac{\left\lfloor \dfrac{\Sigma D \cdot p}{F} \right\rfloor \cdot w_{\mathrm{tgt}}}{F} \right\rfloor \tag{13}$$

2.3 斩杀（溢出）加成

若某一层的伤害分量超过目标**当前船体**（可以单发斩杀）V_{tgt}（目标 $+0xa68$ 是当前船体，上限在 $+0xf20$)，给出 +50% 加成（即乘 1.5)：

$$S_1 = \begin{cases} S_0 + \left\lfloor \dfrac{S_0}{2} \right\rfloor, & V_{\mathrm{tgt}} \le D_2 \\ S_0, & \text{otherwise} \end{cases} \tag{14}$$

2.4 无还手能力降权与模板自定义权重表

$$S_2 = \begin{cases} \left\lfloor \dfrac{S_1\,\kappa_{\text{un}}}{F} \right\rfloor, & \neg\,\text{CanDoDamage(tgt)} \\ S_1, & \text{otherwise} \end{cases} \tag{15}$$

$$S^{(\text{weapon})} = \begin{cases} S_2 + w_k, & \exists\, k:\ \text{key}(k) = \text{key(tgt)} \\ S_2, & \text{otherwise} \end{cases} \tag{16}$$

w_k 来自模板的 `SShipSizeNameTargetWeightPair` 表（表 $+0x910$, 数量 $+0x91c$, 每项 $0x40$ 字节，键 $+0x30$、值 $+0x38$），键为舰船尺寸名。这是" 优先打某类目标" 的脚本入口。

$CanDoDamage$ 仅判断目标的面板 DPD（Damage Per Day，且为舰船设计中显示的数字而不是实际真实数据）是否小于 0

3 调用端：谁在调用、多久调一次

3.1 舰船主目标：`CShipTargeting::CalcBestTarget`（行 1577208）

它枚举敌方候选船、逐个调 `ComputePotentialTargetScore`（行 1577446）拿 $W^{(\text{ship})}$，然后**取最大值**（`argmax`，不是加权抽签）。两个关键行为：

1. **错峰节流**（行 1577268–1577275）：若已有有效主目标，

$$(\text{TotalTicks} + \text{shipID}) \bmod f_p \neq 0 \implies \text{保留当前主目标，跳过整轮权重计算} \tag{17}$$

其中 $f_p = 50$（当 `targetFocus`$= 1$）否则 10；`targetFocus` 取自设计 growth stage $+0x128$（`IsConfused` 时为 1）。

2. M_{cur} 只在 `targetFocus`$= 0$ 时才施加（舰船层与武器层一致）。

3.2 武器选目标：`CWeaponComponentTargeting::CalcBestTarget`（行 1578480）

按顺序：

1. 若组件模板 $+0x12e0 == 2$(舰载机/打击机)→ `CStrikeCraftTargeting::CalcBestTarget`；
2. 若组件标记（模板 $+0x10$ 的 bit2）置位 → `GetShipMainTargetIfValid`：直接沿用**舰船主目标**（前提：在射程与射界内）；
3. 否则 → `CalcBestWeaponComponentTarget`，即每门武器**自己**用 $S^{(\text{weapon})}$ 取最大。

武器模板里有 `use_ship_main_target` 字段；例如 `ENERGY_LANCE`（泰坦光矛）写的是 `no`，所以它走第 3 条，**武器层的分数才是最终决定**——舰船层只影响" 是否存在主目标"。

4 最终计算

对应 `CWeaponComponentTargeting::CalcBestWeaponComponentTarget`（行 1578577）。设候选集合为 $\mathcal{C}$，当前既定目标为 $c_\star$：

$$\tilde{S}(c) = \begin{cases} \left\lfloor \dfrac{S^{(\text{weapon})}(c)\, M_{\text{cur}}}{F} \right\rfloor, & c = c_\star \\ S^{(\text{weapon})}(c), & c \neq c_\star \end{cases} \tag{18}$$

补充：这里的 M_{cur} 只在 `targetFocus`$= 0$ 时生效（判据是 (focus $== 0$) && 有有效主目标，且候选 $==$ 当前主目标）。

$$c^* = \arg\max_{c \in \mathcal{C}} \tilde{S}(c) \tag{19}$$

另外，若候选 c 出现在" 最近目标记忆" 数组 $\mathcal{R}$ （长度 `NUMBER_OF_TARGETS_TO_REMEMBER`）中而**不是**其中最新的一项，则在比较前再施加一次惩罚：

$$\tilde{S}(c) \leftarrow \left\lfloor \frac{\tilde{S}(c)\, M_{\text{rec}}}{F} \right\rfloor, \qquad c \in \mathcal{R} \setminus \{r_0\} \tag{20}$$

其中 r_0 是记忆数组的头部（最近一次的目标）。两个修正都不改变 c^* 之外的目标排序语义。

5 归一化形式（去掉定点除法）

若把定点除法视为精确实数除法，并令 $\hat{W} = W/F$、$\hat{\sigma} = \sigma' \kappa_{\text{size}}/F^2$ 等，可得到便于分析的连续形式：

$$\hat{W}_1 = \frac{1}{1+\hat{\sigma}} \tag{21}$$

$$\hat{W}_2 = \hat{W}_1 \cdot \frac{1}{1+\Delta\kappa_{\text{ev}}/F} \tag{22}$$

$$\hat{W}_3 = \hat{W}_2 \cdot \max\left(1 - \frac{d\,\kappa_d}{F^2},\ \frac{d_{\min}}{F}\right) \tag{23}$$

$$\hat{W}_4 = \hat{W}_3 \cdot \frac{w_{\text{tgt}}}{F} \tag{24}$$

即整体是一个**归一化后的乘法链**：

$$\hat{W}^{(\text{ship})} \propto \underbrace{\frac{1}{1+\hat{\sigma}}}_{\text{尺寸}} \cdot \underbrace{\frac{1}{1+\Delta\kappa_{\text{ev}}/F}}_{\text{闪避}} \cdot \underbrace{\left(1 - \frac{d\,\kappa_d}{F^2}\right)}_{\text{距离}} \cdot \underbrace{\frac{w_{\text{tgt}}}{F}}_{\text{目标权重}} \cdot \begin{cases} \kappa_h/F & h < \theta_h \\ 1 & \text{otherwise} \end{cases} \cdot \begin{cases} \kappa_{\text{un}}/F & \text{无武装} \\ \kappa_{\text{sb}}/F & \text{星基} \\ 1 & \text{otherwise} \end{cases} \tag{25}$$

武器层同理：

$$\hat{S}^{(\text{weapon})} \approx \underbrace{\hat{\Sigma} D \cdot \hat{p}}_{\text{期望伤害}} \cdot \underbrace{\hat{w}_{\text{tgt}}}_{\text{阶段一结果}} \cdot \begin{cases} 1.5 & V_{\text{tgt}} \le D_2 \\ 1 & \text{otherwise} \end{cases} \cdot \begin{cases} \kappa_{\text{un}}/F & \text{无武装} \\ 1 & \text{otherwise} \end{cases} + \hat{w}_k \tag{26}$$

可见阶段一是"目标好不好打/值不值得打", 阶段二是"这门武器打它能造成多少有效伤害", 两者相乘后取最大。

6 定义实测值（`common/defines/00_defines.txt`, $NCombat/NShip$）

符号	define	值
κ_{size}	`COMBAT_SHIP_TARGETING_SIZE_MULT`	0.03
$\sigma_{\max}$	`COMBAT_SHIP_TARGETING_SIZE_MAX`	10
κ_{trk}	`COMBAT_SHIP_TARGETING_TRACKING_MULT`	2.0
κ_{ev}	`COMBAT_SHIP_EVASION_TARGETING_WEIGHT`	2.0
θ_h	`COMBAT_SHIP_LOW_HEALTH_THRESHOLD`	0.5
κ_h	`COMBAT_SHIP_LOW_HEALTH_TARGETING_WEIGHT`	1.5
κ_d	`COMBAT_SHIP_DISTANCE_TARGETING_WEIGHT`	0.002
$d_{\min}$	`COMBAT_SHIP_DISTANCE_TARGETING_WEIGHT_MIN`	0.10
κ_{un}	`COMBAT_SHIP_UNARMED_TARGETING_WEIGHT`	0.005
κ_{sb}	`COMBAT_SHIP_STARBASE_TARGETING_WEIGHT`	0.75
T_{avoid}	`COMBAT_SHIP_EVASION_AVOID_THRESHOLD`	0.7
$\sigma_{\min}^{\text{avoid}}$	`COMBAT_SHIP_EVASION_AVOID_MIN_SHIP_SIZE`	8
κ_r	`RANDOM_TARGETING_WEIGHT`	3.0
M_{cur}	`SHIP_CURRENT_TARGET_MULTIPLIER`	1.1
M_{rec}	`SHIP_RECENT_TARGET_PENALIZER`	0.3
—	`NUMBER_OF_TARGETS_TO_REMEMBER`	10
—	`SHIP_MAX_EVASION`（$NShip$）	0.9
—	`MAX_DAMAGE_SCALING`（$NShip$, 按目标 size 放大伤害的上限）	100

7 数值算例（实测）

两条船、一门 `ENERGY_LANCE`（600–1500, $m_h = 1.5$、$m_s = 0.5$、$m_a = 2.0$, `acc`= 0.85、`trk`= 0）:

	护卫舰	雷击舰
σ（size_multiplier）	5.00	8.00
ε（闪避）	0.600	0.424
船体 / 盾 / 甲	240/225/225	720/0/0
d	72.0	64.5
w_{tgt}（本场实录）	1.561	1.379

W_1 尺寸	0.8696	0.8065
W_2 闪避	0.4545	0.5411
W_3 距离	0.856	0.871
$W^{(\text{ship})}/W_4$	**0.528**	**0.524**
p（命中率）	0.250	0.426
ΣD 相对值	2.648	1
$S^{(\text{weapon})}$ 相对值	**1.76**	**1.00**

结论：**舰船层两条基本持平**(0.99)，**武器层护卫舰高** 1.76×。因为 ENERGY_LANCE 的 use_ship_main_target = no，实际按武器层走，所以会优先打带盾甲的护卫舰——原因不是" 盾甲吸引仇恨"，而是 ΣD 被盾甲顶高了 2.65×，盖过了它高闪避带来的 p 劣势（1/1.70）。

附 A：关键函数地址（构建 hook 用）

符号（mangled）	说明
_ZN14CShipTargeting14CalcBestTarget…	舰船选主目标（argmax + 节流）
_ZN14CShipTargeting27ComputePotentialTargetScore…	舰船层权重（出参 RCX）
_ZN25CWeaponComponentTargeting27ComputePotentialTargetScore…	武器层分数（返回值）
_ZNK5CShip13ApplyModifier…	把修正套进 SAttackInfo
_ZN12NCombatRules10CalcDamage…	三层伤害分量
_ZN12NCombatRules15CalcChanceToHit…	命中率

附 B：常量来源（行号）

常量	出现位置
尺寸项 W_1	1577500–1577533
追踪–闪避项与否决条件	1577538–1577690
距离项 W_3	1577775–1577800
目标预置权重 W_4	1577805
残血项 W_5	1577833–1577855
无武装/星基项 W_6	1577856–1577890
随机项	1577896–1577935
w_{tgt} 来源	3824970（取值）/ 3830406（开战随机）
阶段二伤害链	1578737–1578935
斩杀 / 无武装 / 自定义表	1578815–1578925
最终计算（当前目标、近期目标）	1577290–1577360

附 C：伤害计算 `NCombatRules::CalcDamage`

武器层用来算” 这一发打在该目标上大概能造成多少伤害” 的函数。它输出三个伤害分量，武器层把三者相加得到 ΣD（$\Sigma D = D_1 + D_2 + D_3$）。

签名（`objdump` 核对的寄存器）：

```
CalcDamage(
    SDefenseInfo const& def,                                  // RSI
    SAttackInfo const& atk,            // RDX（已被 ApplyModifier 改写）
    SShipDamageReductionInfo const& red,                      // RCX
    bool useRandom)                     // R8；返回值经 sret(RDI) 写回
```

目标防御 `SDefenseInfo`（由 `CShip::BuildDefenseInfo` 填充）

off	含义	来源
$+0x00$	当前护盾	$ship + 0xa70$
$+0x08$	当前装甲	$ship + 0xa78$
$+0x10$	`size_multiplier`	$ship + 0x630$
$+0x18$	护盾硬化	船级/修正累计
$+0x20$	装甲硬化	船级/修正累计

武器 `SAttackInfo`（偏移以模板 $+0x11c8$ 为 0）

off	字段	`weapon_components.csv` 列
$+0x00$	$D_{\min}$	min_damage
$+0x08$	$D_{\max} - D_{\min}$（跨度，不是上限）	max_damage − min_damage
$+0x10$	m_h	hull_damage
$+0x18$	m_a	armor_damage
$+0x20$	m_s	shield_damage
$+0x28$	装甲穿透	armor_penetration
$+0x30$	护盾穿透	shield_penetration
$+0x38$	`size_damage_factor`	size_damage_factor
$+0x40$	军力倍率	military_power_multiplier

（字段布局取自该 CSV 的读取器,行 3878100–3878210;$+0x11a8$ / $+0x11b0$ 上分别挂着”Accuracy / Tracking should be between 0.0 and 1.0” 的报错文案，可反查命中率与追踪率。）

计算步骤

1. **基础伤害**（索敌路径传 useRandom = false，取区间中点）：

$$D = D_{\min} + \tfrac{1}{2}(D_{\max} - D_{\min}), \qquad D \leftarrow D\,(1 - r_1), \qquad D \leftarrow \max(D - r_0,\ 0)$$

其中 r_0, r_1 为传入的减伤信息；**武器层调用时恒为** $(0, 0)$，即估算伤害不减伤。

2. **按目标尺寸放大**（仅当 $f_{\text{size}} > 0$）：

$$sz = \text{clamp}(\sigma,\ 1,\ 100), \qquad D \leftarrow D\,(1 + sz\, f_{\text{size}})$$

f_{size} 即 `size_damage_factor`，100 为 `MAX_DAMAGE_SCALING`。

3. **护盾层**（当前护盾 > 0 才执行）：

$$s = \text{clamp}(p_s - h_s,\ 0,\ 1), \quad C_s = (1 - s)\, D, \quad D \leftarrow D - \min\big(C_s,\ \text{护盾}\big), \quad D_s = m_s\, C_s$$

p_s = 护盾穿透、h_s = 护盾硬化、m_s = `shield_damage`。

4. **装甲层**（当前装甲 > 0 才执行）：

$$a = \text{clamp}(p_a - h_a,\ 0,\ 1), \quad C_a = (1 - a)\, D, \quad D \leftarrow D - \min\big(C_a,\ \text{装甲}\big), \quad D_a = m_a\, C_a$$

p_a = 装甲穿透、h_a = 装甲硬化、m_a = `armor_damage`。

5. **船体层**：剩下的全给船体，

$$D_h = m_h\, D$$

m_h = `hull_damage`。

其中 D 在每一层都被 $\min(\cdot)$ 扣减，最后一格 D 即" 剩余伤害"。

返回值（sret）

off	含义
$+0x00$	标志（$= 1$）
$+0x08$	D_h（船体分量）$\leftarrow$ **斩杀加成判据用的就是它**
$+0x10$	D_a（装甲分量）
$+0x18$	D_s（护盾分量）
$+0x20$	剩余伤害（扣完各层后）
$+0x28$	C_a（装甲层分量，未乘倍率）
$+0x30$	C_s（护盾层分量，未乘倍率）

要点

- **层分量不被该层存量夹住**：C_s, C_a 由" 剩余伤害" 折算而来，可以远大于该层当前的盾/甲量；只有从剩余伤害里扣减的量才被 min 夹住。因此**带护盾/装甲的目标算出来 ΣD 反而更大**——这就是" 带盾甲的船更容易被选中" 的根因。
- **穿透减去硬化后夹到** $[0, 1]$，再用 $1-$ 作为" 保留比例"；m_s, m_a, m_h 只是三层各自的输出倍率，三者互相独立。
- 传入的 `SAttackInfo` 是**已套修正**的版本，所以 ΣD 用的是修正加成后的伤害（含舰船/武器/目标相关加成）。
- 船体层拿到的是" 扣完盾甲后的剩余伤害",所以只有" 无盾无甲" 的目标,ΣD 才等于 $m_h \cdot D$。